References and registries (L3)

TBK SPEC 04:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 04:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword ____ 4

1. The reference syntax (MECE, no default service) ____ 4

2. The registry (developer-hosted, zero central infrastructure) ____ 6

3. Fetch discipline ____ 9

3a. Run-time payload resolution grammar (tebako#460) ____ 10

4. Enterprise networking — proxies and trust anchors (2026-09-06 amendment) ____ 10

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative specification of how payloads are named, located, and listed. Status: syntax LOCKED (multi-artifact + repo/release-class rules locked 2026-07-26); resolver + registry listing + `tebako install` SHIPPED (roadmap 07/28.1); `tebako publish` SHIPPED (roadmap 41 — GitHub releases + file:// mirrors; the GitLab/Bitbucket write legs and the brew/install.sh channels remain PLANNED).

1. The reference syntax (MECE, no default service)

One scheme family; the adapter is explicit in the scheme, never inferred:

FIGURE 1

```
SERVICE adapters (the service's ARTIFACT storage — never the
git tree):
  tfs:github:owner/repo:version[#artifact]   → GitHub
releases API
  tfs:gitlab:owner/repo:version[#artifact]   → GitLab
releases/packages API
  tfs:bb:owner/repo:version[#artifact]       → Bitbucket
downloads API

PROTOCOL adapters:
  tfs+git://host/owner/repo.git@<ref>#<path-in-repo>
      (the GIT SOURCE — any host incl. github/gitlab/
bitbucket, LFS-aware;
       #path selects the file when a repo holds many images;
optional when
```


© 2026 The Tebako Project

```
          the repo IS the registry)
      tfs+https://cdn.example.com/tool.tfs → verbatim fetch

    LOCAL:
      file:///opt/images/tool.tfs

    DIGEST PIN (any class, query form — never clashes with
    #artifact/#path):
      ...?sha256=<hex>
```

Repo vs releases is a class distinction, never a flag. The git tree is addressed ONLY by `tfs+git:`; release artifacts ONLY by `tfs:<service>:`. Both are supported by design — a service form never means "the repo", a git form never means "the release".

Multi-artifact releases (locked — no magic):

- `#artifact-name` selects one artifact within the release (same # convention as `#path-in-repo`).
- No #: the candidate class is `.tfs` images. Exactly ONE candidate → used (there is no choice to guess). ZERO → `AssetNotFound`. MORE THAN ONE → `AmbiguousAssets` naming every candidate so the user re-runs with `#name`. The adapter NEVER auto-picks by host triplet — platform selection is the registry's declarative job (§2).

Dispatch rule (deterministic, exactly one home per reference):

TABLE 1

scheme	adapter
`tfs:github:` / `tfs:gitlab:` / `tfs:bb:`	that service's releases API
`tfs+github:///` / `tfs+gitlab://` (spec 37 §4)	that host's releases API (GHE / self-hosted)
`tfs+git:`	git protocol adapter
`tfs+oci://` (spec 38 §2)	the OCI Distribution registry
`tfs+https:` (or bare http(s))	http fetch
`file://`	local file
`tfs+bb:///` `tfs+bitbucket://` (Data Center)	the named `UnsupportedService` refusal — never a guess
`tfs+git+ssh://`	the named `SshTransportUnsupported` refusal
anything else	named error listing the classes

There is **NO default service or namespace** anywhere: every reference is fully explicit. (Superseded forms — `tfs://github.com/owner/repo:version` and any host-inferred shorthand — are rejected, never guessed.)

© 2026 The Tebako Project

Manifest `requires` entries (spec 03) resolve through `{ref, constraint, mount}`.

2. The registry (developer-hosted, zero central infrastructure)

A **registry** is ANY git host repo carrying `tpkg-registry.yaml`. The git host's releases ARE the storage — no server to run.

Registry resolution (locked): `tfs:<service>:owner/repo` (no version) resolves the registry file from the repo's DEFAULT-BRANCH ROOT via the service contents API (`/tpkg-registry.yaml`); the pinned-immutable form `tfs:<service>:owner/repo:version#tpkg-registry.yaml` reads it as a release artifact (versioned with its payloads); `tfs+git://...#path` reads it from any git ref/path. No other locations, no search, no fallback chain — exactly one location per form.

FIGURE 2

```
schema_version: 1
payloads:
  - name: metanorma
    kind: app
    versions:
      - version: 1.2.3
        platforms:                    # per-triplet for
native-ext apps;
          x86_64-linux-gnu:           # "universal" for pure-
language
            artifact: metanorma-1.2.3-linux-gnu-x86_64.tfs
            sha256: "…"
          aarch64-macos:
            artifact: metanorma-1.2.3-macos-arm64.tfs
            sha256: "…"
        release: {ref: tfs:github:metanorma/metanorma:1.2.3}
        signature: {keyid: "…", asc: "…"}        # opt-in
        runtime_requirement: {engine: ruby, constraint: "~>
3.3.0"}
        entrypoints: [metanorma]
    default: 1.2.3
```

- **Mirrors on the row (additive, shipped 2026-10-22; registry MINOR 10):** a version entry, or one of its per-triplet `platforms` entries, may carry `mirrors:` — an ordered list of ordinary references naming other places that serve the SAME bytes. The row's `sha256` remains the only anchor: the primary is tried first, each mirror afterwards, and every attempt is verified against that pin, so a mirror can only serve the same bytes or

© 2026 The Tebako Project

fail by name. The anchor is computed ONCE from the primary (the registry pin, else the primary's own `?sha256=`): a mirror's own ref never replaces or weakens it. A single-locator row that fails keeps its own error class and wording; a chain that is exhausted fails with every attempted locator listed, carrying the most specific refusal class (a trust mismatch outranks plain unavailability). This does not soften the locked rule above: a REFERENCE still names exactly one location and there is still no resolver-side search; the list lives in the document the publisher authored, and it is read only after the primary fails. `tebako registry validate` names a mirror list on a `universal` row with no pin, a list on a per-triplet row at the version level, an empty list, and a mirror whose reference does not parse, as producer-gate violations.

- The registry MIRRORS only resolution-relevant fields (spec 03 §4 tier 3) — the dispatcher resolves without downloading every payload.
- **The mirror's `runtime_requirement.abi` rule (additive, 2026-10-07, tebako#440):** the abi is per-triplet by construction, so a version entry's mirrored requirement carries `abi` only when one value holds for the WHOLE entry (a `universal` row or a single-platform per-triplet row). A multi-platform per-triplet entry omits it — one platform's abi on an entry serving every triplet is wrong for every platform but one. The authoritative per-platform abi always lives in each slice's embedded manifest; dispatch reads it there, never from the mirror. `tebako registry validate` names a multi-platform row's `abi` a producer-gate violation (2026-10-08) — the reader stays lenient so already-published registries keep resolving.
- **The variant dimension (additive, 2026-10-09, tebako#557; spec 28 §2–§3):** a version entry carries EITHER the shorthand above (top-level `platforms:` + `runtime_requirement:` — exactly the single-variant form) OR `variants:` — a list of `{runtime_requirement, platforms}` arms, one per (runtime_requirement × platforms) build of the same version — never both. Each variant is keyed by its requirement alone (the variant id derives, spec 28 §2); a duplicate key inside one version, both forms at once, or a `default_variant:` (payload level) naming no variant of the default version are named registry validation errors. With no selector in force, resolution picks the declared `default_variant`, else the variant whose requirement's line is newest (spec 28 §4 rule 1). Pre-variant readers see a variants-only entry as a version with no top-level `platforms:` and refuse it loudly — a clean break, never a mis-resolution.
- **The per-row shard `release.ref` (additive, 2026-10-07, tebako#711):** a `platforms[<triplet>]` entry may carry its own `release: {ref: …}` naming the shard release tag serving THAT row's bytes. A version line that unions rows from several per-platform shard tags (the runtime factory's shape) cannot name the serving tag at the version level — the row carries it. The same grammar discipline as the version-level ref under per-triplet platforms applies (no `#artifact`, no `?sha256=` pin, a service release); MINOR-6 readers prefer the row's ref over the version-level one, pre-MINOR-6 readers ignore the key (the version-level ref must then name a tag serving every row — the pre-shard shape).
- **The head `signing:` block (additive, 2026-10-07, tebako#617):** the registry's signing-key self-description (spec 09 §9.1) — `key:` (armored OpenPGP public key), `fingerprint:` (the key's own primary, 40 hex), optional `url:` (https cross-check). `tebako add-registry` verifies the block against its own key, then TOFU-pins it

© 2026 The Tebako Project

(prompt, or `--expect-fingerprint` unattended); pre-MINOR-7 readers ignore the block.

- `status: withdrawn` (additive, 2026-09-12, roadmap 85): the entry is YANKED — resolvers refuse it with a named `WithdrawnPayload` error (never a silent skip, never a fallback to it), and listing surfaces render it as withdrawn. Release assets are immutable (spec 13 §2a's immutability rule): withdrawal is the ONLY remedy for a bad published artifact, and the fix ships as a new version line.
- Host-triplet selection happens HERE, declaratively: the dispatcher reads `platforms[host_triplet].artifact` (or `universal`), fetches THAT artifact from the named release. Never adapter-side guessing.
- Concrete rows only (locked 2026-08-25, tebako#460). A version carries a `universal:` row OR per-triplet rows — never patterns (`linux-*-*` is rejected at validation), never both. Identical bytes under multiple rows dedup naturally (content-addressed cache). A composition's `platforms:` assertion (spec 23 §13.3) CONSTRAINS the mirrored coverage — checked fail-closed against it — and NEVER extends it.
- `kind: runtime` entries are edge-discoverable (schema MINOR 1; SHIPPED 2026-09-25, spec 37 §8): the entry carries `engine:` (+ optional `implementation:`), and a payload's `kind: runtime` DEPENDS edge (spec 30 §1) discovers it by (engine, implementation?, constraint); the matched version's `release.ref` derives the download base (spec 05 §2's per-engine chain, the zero-config path for third-party runtimes). Runtime entries predating MINOR 1 carry no `engine:` and stay invisible to edges. The payload-level spelling is the one authored grammar (2026-10-08, tebako#549, MINOR 8): the version-level `implementation:` remains a compat READ (MINOR 2 — already-published registries keep resolving) but is never authored anew, and a version-level `engine:` parses to nothing; `tebako registry validate` names the old spelling, a mix of both spellings, and the misplaced `engine:` key as producer-gate violations, so a registry PR fails its own CI leg instead of publishing a decorative axis.
- Bundle-era runtime rows name the bundle (additive, 2026-09-24, spec 36): `platforms[<triplet>].artifact` is `<stem>.tar.gz` and the row's `sha256` pins the bundle, whose members unpack into the runtime store (spec 36 §4). Per-file-era rows keep naming the exe — the mirror is faithful to what the pinned release serves, era by era, and `status: withdrawn` means the same on both.
- `tebako add-registry <ref>` registers one; shipped config has ZERO registries (explicit only — spec 16).
- Install = resolve the registry → select the host entry → download → verify (sha256 and/or OpenPGP signature, spec 09) → content-addressed cache (`~/.tebako/payloads/<name>/<version>.tfs`) → register shims (spec 07). An install whose selected entry is `kind: runtime` refuses BEFORE any download (2026-10-07, tebako#672): the named `RuntimeKindNotInstallable` error (exit 65) points at the working paths — `tebako install <the app that needs it>` (its runtime_requirement resolves the runtime) or `tebako use --runtime <engine>@<langver>[:<tebako>]`. Runtimes flow through the runtime cache, never the payload store.

© 2026 The Tebako Project

- A developer's release flow: press → sign → upload payloads → commit `tpkg-registry.yaml` (a `tebako publish` helper later; manual first).

3. Fetch discipline

- In-process HTTP only (spec 01): ureq + rustls + webpki-roots bundled; OS roots opt-in via `TEBAKO_TLS_PLATFORM_ROOTS`; `file://` mirrors for tests and air-gapped sites.
- Git protocol via `gix` (gitoxide) when `tfs+git:` needs smart-protocol fetch — never the git CLI.
- The capability is per-binary, not per-ecosystem. The shipped `tebako-bootstrap` (the 3 MiB-gated loader) links tebako-resolve with the `git` feature OFF: `tfs+git:` lock sources fail closed with the named `GitAdapterDisabled` error (steer to managed mode — `tebako install` / the shim — or mirror to `tfs+https:`). The toolchain (tebako CLI, tebako-shim, tfs CLI) always carries the full surface. General rule: an optional capability is a cargo feature on the owning crate — default ON for the toolchain, OFF for the bootstrap, named error when compiled out.
- Atomic fetch: download to tmp, verify digest, rename; concurrent fetchers coordinate via the cache lock (spec 05); a partial fetch is invisible. `TEBAKO_OFFLINE=1`: cache hit or hard error.
- Private GitHub repos (2026-09-15 amendment): with `TEBAKO_GITHUB_TOKEN`/`GITHUB_TOKEN` ambient, the GitHub adapter resolves assets to their API URL (`/repos/{o}/{r}/releases/assets/{id}`) instead of `browser_download_url` — private repos answer 404 on the anonymous browser URL. The fetch requirements the choice implies are declared ON the asset descriptor the adapter returns (`accept` — the API serves JSON metadata without `application/octet-stream`; `authenticate` — credential-eligibility), never inferred from URL text downstream; the bearer rides `api.github.com` ONLY (never the browser or CDN hosts, never a redirect target). Anonymous transports keep the browser URL: the CDN path spends no API rate budget and carries no credential. A private asset with NO token is the ordinary named `NotFound`, never a fallback chain.
- Digest mismatch → the named sha error (exit 70); nothing enters the cache.

© 2026 The Tebako Project

3a. Run-time payload resolution grammar (tebako#460)

The L2 package manifest's resolution rows carry the SAME reference grammar as §1 plus the press-locked pin: `ref` (the registry/source reference), `version` (the locked concrete version), and `sha256` (the locked digest). Run-time resolution consumes the locked pair ONLY — the ref locates, the digest decides; a version resolved fresh that disagrees with the lock is the trust-error class, never a silent swap. Registries, offline mode, and the fetch discipline (§3) are identical with the runtime's own chain.

4. Enterprise networking — proxies and trust anchors (2026-09-06 amendment)

Every fetch the loader or the toolchain makes rides ONE client (`tebako-http::build_agent`); the policy below is resolved once per process from the environment merged over the `network:` section of `~/.tebako/config.yaml` — env wins per key — and installed before the first request (agent construction caches the transport). The resolution is journaled (`event=network-config …`) with proxy credentials REDACTED (`http://***@host:port`). There is no verify-off spelling, and there never will be.

Proxy.

- Env: `HTTPS_PROXY` / `HTTP_PROXY` / `ALL_PROXY` / `NO_PROXY` (both cases), the de-facto grammar: comma list, suffix match, *; localhost is always direct. Config mirror: `network.proxy: <url>`.
- Precedence: env → config → direct. Credentials ride the URL (`http://user:pass@host:port`) — the only auth spelling.
- HTTPS rides CONNECT through the proxy; TLS terminates at the TARGET. An inspecting proxy re-encrypts — that is what the trust half below is for.
- Named errors, never a silent fallback: `ProxyUrlInvalid`, `ProxySchemeUnsupported` (no SOCKS), `ProxyAuthRequired` (a 407 the URL's credentials did not satisfy). Configuration errors are deterministic: retry loops return them immediately, never as `IndexUnavailable`, never retried.

Trust anchors — three spellings, all fail-closed:

1. `webpki` (default, unchanged): the bundled Mozilla roots.

© 2026 The Tebako Project

2. `platform`: the OS store (GPO/MDM-pushed enterprise roots) — env `TEBAKO_TLS_PLATFORM_ROOTS` (since v2.0.0) or `network.tls_roots: platform`.
3. Additive: `network.extra_ca: [<pem>…]` / `TEBAKO_EXTRA_CA` (os-path-list separator) — the PEMs parse at client build INTO the bundled store (never instead of it). Combining `platform` with `extra_ca` is the named `ExtraCaWithPlatformRoots` error (the OS verifier trusts exactly the OS store — push the CA there instead). A malformed or unreadable PEM is `ExtraCaMalformed` / `ExtraCaUnreadable` at startup — never a silently skipped file.

The eMudhra distrust policy (2026-10 amendment). eMudhra's intermediate CA private keys were generated with recoverable entropy (one key is literally the integer 4). tebako excludes the CA outright from every trust set this stack assembles:

TABLE 2

Assembled set	Owner	Distrust effect
bundled Mozilla roots (`webpki` / default)	tebako-http `distrust`	every eMudhra/emSign root is dropped before install; `RootCerts::WebPki` is never used (it would bypass the filter)
additive `extra_ca` PEMs	tebako-http `netconfig`	a file carrying a distrusted cert is the named `ExtraCaDistrusted` — never a silent drop
operator CA store (`$TEBAKO_HOME/trust/ca`)	tebako-driver `materialize`	a store file carrying a distrusted cert is a named 65 naming `tebako trust remove`
driver platform enumeration (merged runtime bundle)	tebako-driver `materialize`	every distrusted DER is dropped before the merge; the drop is debug-journaled; a store that reduces to zero is a named 65

Identity is lexical over the certificate DER (subject/issuer DN markers `eMudhra` / `emSign`, case-insensitive) — it survives re-issues, cross-signs, and the Microsoft-only `emSign Root TLS/CS CA - G3` pair. The compromised intermediates (including the AAA-cross-signed ones) are additionally blocked at verification time by rustls-webpki ≥ 0.103.16's SPKI blocklist; the bundled-root floors are `webpki-root-certs` / `webpki-roots` ≥ 1.0.10 (which drop `emSign ECC Root CA - G3` upstream). Platform mode trusts the OS store as-is at the TLS verifier (the OS owns its own distrust); the driver still filters the enumerated blocks it merges into a runtime bundle.

Capability gating (§3's general rule applied): `network` is a cargo feature on tebako-http — default ON for the toolchain, OFF for the size-gated bootstrap. Compiled out, a set proxy env or `TEBAKO_EXTRA_CA` fails closed with the named `NetworkingCompiledOut` error identifying the variable; the bootstrap never silently ignores a policy the operator asked for. `TEBAKO_TLS_PLATFORM_ROOTS` is NOT part of the feature — the platform-verifier opt-in shipped in v2.0.0 and stays always-on in every build, bootstrap included. The config mirror is read by the toolchain (shim / CLI) at startup; the size-gated bootstrap reads env policy

© 2026 The Tebako Project

only — enterprise fleets push policy as system env (GPO/MDM), which both channels honor.

FIGURE 3

```
# ~/.tebako/config.yaml — every key optional; env wins per
key
network:
  proxy: http://user:pass@proxy.corp:3128  # CONNECT proxy
  tls_roots: platform                       # default: webpki
  extra_ca: [/etc/pki/corp-root.pem]        # additive; not
with platform
```

The payload side (a packaged Ruby/Python process making its OWN TLS connections) is out of the loader's scope: it rides the runtime's `SSL_CERT_FILE` semantics — spec 22 §4.

© 2026 The Tebako Project